

PTE

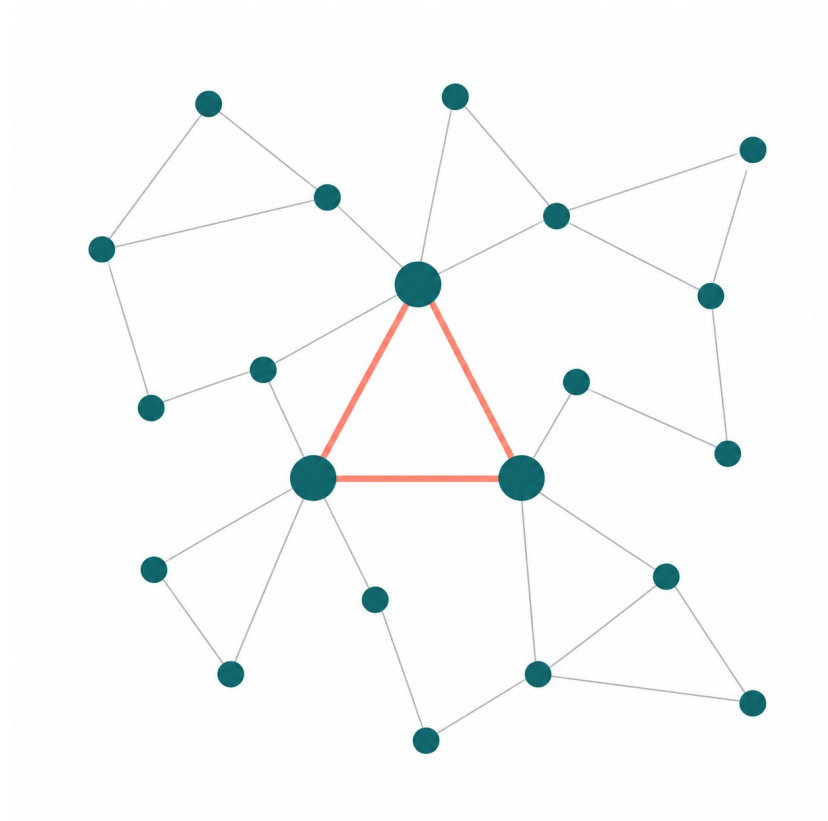
대규모 그래프의 삼각형 열거

중간 데이터 병목과 Pre-partitioned Triangle Enumeration

Ha-Myung Park, Sung-Hyon Myaeng, U Kang

KDD 2016, pp. 1115-1124

PTE: Enumerating Trillion Triangles On Distributed Systems



Triangle enumeration의 의미

서로 모두 연결된 세 정점을 하나씩 발견한다

세 정점과 세 간선

정점 A, B, C

간선 $(A, B), (A, C), (B, C)$

무방향 단순 그래프

자기 루프·중복 간선 없음

발견한 (A, B, C) 는 $\text{enum}(A, B, C)$ 로 처리한다.

논문의 문제 정의는 모든 결과를 파일에 저장하도록 요구하지 않는다.

Counting

삼각형이 몇 개인지 센다

Enumeration

각 삼각형의 구성 정점을 발견한다

그래프가 커질 때 생기는 병목

ClueWeb12: 입력뿐 아니라 중간 데이터와 결과도 커진다

63억

정점

720억

간선

약 3.06조

삼각형

RAM

지금 계산하는 부분 그래프와 작업 버퍼

SSD / HDD

입력 파일, 분할 파일, 임시 중간 파일

네트워크

다른 장비로 옮기거나 읽어 오는 데이터

삼각형을 모두 저장하면 원문 추산 약 70 TB.

이 저장 비용과 계산 중 발생하는 중간 데이터는 서로 다른 문제다.

External memory는 RAM이 아니다

책상(RAM)과 책장(SSD/HDD), 완성한 결과의 보관 공간을 구분하자

자원	담는 것	부족하면 생기는 일
Main memory RAM	지금 계산할 간선·인덱스·버퍼	작업을 더 작게 나누거나 메모리 부족으로 중단
External memory SSD / HDD	입력과 분할 파일·임시 파일	한 머신의 저장공간을 넘으면 필요한 파일을 둘 곳이 없음
Output storage 결과 저장소	모든 triangle을 남길 때의 정점 ID 세 개씩	입력 처리와 별개로 결과 보관 용량이 필요

$3.06\text{조} \times 3\text{개 ID} \times 8\text{ byte} \approx 73.5\text{ TB}$: 단순 저장 형식의 계산 예시.
논문의 ‘약 70 TB’는 결과 저장 규모이며, RAM 요구량이 아니다.

입력보다 중간 데이터가 더 커지는 이유

정답이 적어도, 후보와 복제본은 많을 수 있다

- | | | |
|---|------------|---|
| 1 | 후보 폭증 | 한 중심에 이웃 100만 개: 간선 약 100만 개.
이웃 쌍을 모두 만들면 후보는 약 5천억 개다. |
| 2 | 간선 복제 | 삼각형을 놓치지 않도록 같은 간선을
여러 부분 문제의 입력으로 보낼 수 있다. |
| 3 | Shuffle 비용 | 복제 레코드 생성 → 임시 디스크 쓰기 → 전송.
받는 쪽의 읽기·병합·정렬 비용도 함께 생긴다. |

별 모양 그래프는 triangle이 0개여도 후보가 매우 많을 수 있다.
모든 알고리즘이 이 후보들을 실제 저장한다는 뜻은 아니다.

기존 알고리즘의 접근과 한계

처리 규모의 한계는 방법마다 다르게 나타난다

접근	해결하려는 문제	남는 병목
단일 머신 I/O-efficient	RAM보다 큰 그래프를 디스크에서 나눠 읽기	한 머신의 저장공간과 반복 디스크 I/O
Distributed memory	여러 머신의 RAM에 그래프를 나누어 보관	복제된 그래프와 중간 데이터가 분산 RAM 용량을 초과
GP / TTP	부분 문제들을 MapReduce로 병렬 처리	부분 문제마다 간선을 복사해 대량의 shuffle 발생
CTTP	여러 round로 나누어 round당 데이터량 제한	전체 shuffle 양은 여전하고 round마다 입력을 다시 읽음

이 표는 데이터 이동 관점의 요약이다. CTTP의 해시·부하 분석은 PTE와 별도로 구분한다.

GP 대비 TTP의 Constant-factor 개선

같은 색 수 $\rho \geq 3$, 균등 무작위 색 배정. m 은 입력 간선 수

Shuffle 간선 레코드 수	GP	TTP
같은 색 끝점의 간선 간선당 레코드 수	$(\rho - 1)(\rho - 2) / 2$	$\rho - 1$
서로 다른 색 끝점의 간선 간선당 레코드 수	$\rho - 2$	$1 + (\rho - 2) = \rho - 1$
전체 그래프 기대 레코드 수	$3m(\rho - 1)(\rho - 2) / (2\rho)$	$m(\rho - 1)$

$\rho > 6$ 에서 TTP의 기대 레코드 수가 감소. 큰 ρ 에서 $TTP / E[GP] \approx 2/3$.
두 방법 모두 $\Theta(m\rho)$. 레코드 수의 상수 계수 개선이며 실행시간 비율은 아니다.

분할 기준 1: 정점에 색 라벨을 붙인다

같은 주제·커뮤니티·차수로 묶는 것이 아니라, 해시로 배정한다

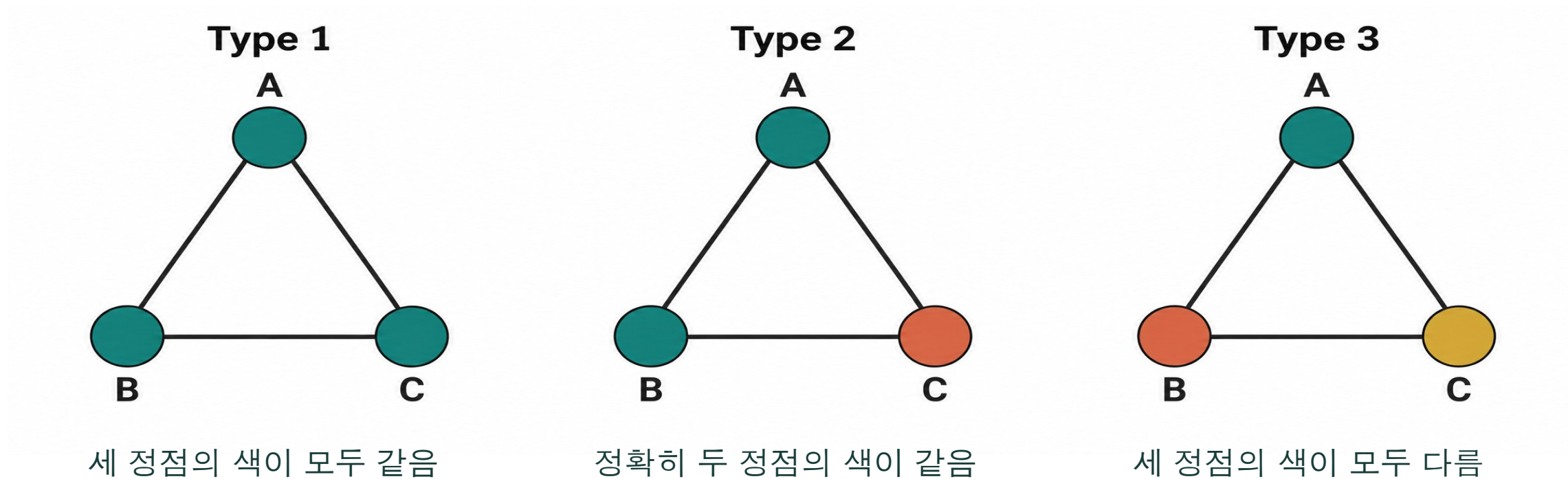
- 1 함수 하나 선택 무작위로 고른 함수 ξ 가 각 정점 v 에
0부터 $p - 1$ 까지의 색 $\xi(v)$ 를 돌려준다.
- 2 실행 중 고정 같은 정점은 어디서 읽어도 같은 색이다.
간선을 만날 때마다 색을 다시 뽑지 않는다.
- 3 작은 예시 서로 다른 정점 A, B, C에 색이 0, 0, 1이면
A와 B의 라벨만 같다. 정점이 합쳐지지 않는다.

p 는 색의 개수이지 머신 대수가 아니다. 간선 수와 작업별 RAM을 보고 정한다.

ID의 단순 나머지 연산을 논문의 실제 해시 구현이라고 가정하지 않는다.

색상으로 나누는 세 가지 triangle type

TTP·CTTP·PTE가 공유하는 분류 기준 — CTTP만의 고유 기능은 아니다



A, B, C는 서로 다른 정점이며, 세 간선이 이미 있어야 triangle이다.

색은 라벨일 뿐이다. 색을 바꿔도 원래 그래프의 간선이나 triangle은 변하지 않는다.

분할 기준 2: 색 조합이 하나의 작업이 된다

각 작업은 두 색 또는 세 색 사이의 간선만 다룬다

부분 문제	읽을 간선 묶음	찾을 삼각형
두 색 (i, j)	i-j 사이, i 내부, j 내부	Type 1과 Type 2
세 색 (i, j, k)	i-j 사이, i-k 사이, j-k 사이	Type 3

A, B, C의 색이 0, 0, 1인 triangle $\rightarrow$ 두 색 작업 (0,1).

색이 0, 1, 2라면 $\rightarrow$ 세 색 작업 (0,1,2).

전체 두 색·세 색 조합을 작업으로 만든다. 한 머신은 여러 작업을 맡을 수 있다.

Type 1은 여러 두 색 작업에 포함되므로 중복 출력 방지 규칙이 필요하다.

분할 기준 3: 간선 파일은 끝점 색의 순서로

정점 순서와 색 번호의 크기는 서로 다른 기준이다

같은 예시의 두 간선	끝점의 색	실제 저장 파일
정점 순서: A가 C보다 앞	A=0, C=1	E_star[0,1]
정점 순서: C가 B보다 앞	C=1, B=0	E_star[1,0]

E01: 색 0과 색 1을 잇는 간선 집합 (0·1은 색 라벨)

$$E01 = E_star[0,1] \cup E_star[1,0]$$

정점 순서는 차수 $\rightarrow$ 동률이면 ID로 정한다. 예시는 $A < C < B$ 를 가정한다.

Algorithm 1의 key를 색 번호의 min/max로 바꾸면 방향 정보가 사라진다.

TTP에서는 간선 하나가 여러 작업으로 복제된다

색이 네 개(0,1,2,3)일 때, 한 간선의 행선지를 세어 보자

간선 끝점의 색	보내는 작업	복제 레코드
0, 0	두 색: (0,1), (0,2), (0,3)	3개
0, 1	두 색: (0,1) 세 색: (0,1,2), (0,1,3)	3개

색이 p 개이면 간선마다 $p - 1$ 개 레코드를 보낸다.

작은 문제를 많이 만들수록, 같은 간선을 더 많이 배송한다.

CTTP는 작업을 여러 round로 나누지만 전체 복제 레코드 수는 남는다.

이 표는 TTP의 복제 예시이며, 색이 4개일 때 GP보다 낫다는 비교가 아니다.

PTEBASE: 분할 저장과 열거의 분리

간선을 미리 분류해 저장하고, 필요한 묶음을 직접 읽는다

- 1 한 번 분할** 간선마다 색상쌍 key 하나를 만든다.
MapReduce로 묶어 HDFS의 분할 파일에 저장한다.
- 2 작은 작업 배정** 두 색 또는 세 색 조합을 작업으로 만든다.
각 map task는 필요한 분할 파일을 직접 읽는다.
- 3 RAM에서 열거** 작업 안에서 삼각형을 찾아 enum을 호출한다.
이 열거 단계에는 별도의 shuffle / reduce가 없다.

각 입력 간선은 분할 단계에서 한 번만 emit한다.
동일한 분할 파일을 여러 작업이 읽는 비용은 여전히 남는다.

Pairwise independence: 왜 평균적으로 퍼질까?

간선 하나의 목적지는 두 끝점의 색으로 결정된다

단계	색이 3개일 때	의미
간선 하나	특정 순서쌍의 확률 $1/3 \times 1/3 = 1/9$	두 끝점의 색이 균일하고 서로 독립
간선 900개	각 $E_{\text{star}}(i,j)$ 에 기대상 $900/9 = 100$ 개	간선별 확률을 더한 평균 실행 측정값은 아님

E_{01} 은 색 0과 색 1 사이의 간선 집합이다.

$E_{\text{star}}(0,1)$ 과 $E_{\text{star}}(1,0)$ 을 합쳐 기대상 $100 + 100 = 200$ 개.

E_{00} 은 색 0 내부 간선: $E_{\text{star}}(0,0)$ 하나이므로 기대상 100개다.

두 집합을 합쳐도 간선 중복은 없다. 기대값은 매 실행의 실제 개수를 보장하지 않는다.

색은 몇 개로 나눌까? RAM과 읽기 비용의 균형

작업 입력의 기대 크기가 RAM 예산 안에 들도록 색 수를 정한다

- | | | |
|---|------------|--|
| 1 | 간선 수 m | 입력이 클수록 같은 색 수에서
각 작업이 읽어야 할 간선도 많아진다. |
| 2 | 작업 RAM M | 세 색 작업은 서로 다른 색 사이의 세 묶음을 읽는다.
기대 크기 $6m/\rho^2 \leq M$ 이 되도록 정한다. |
| 3 | 색 수 ρ | PTEBASE: $\rho = \text{ceil}(\text{sqrt}(6m/M))$.
색을 늘리면 작업은 작아지지만 작업 수·반복 읽기는 늘어난다. |

m 과 M 은 간선 수와 그에 대응하는 메모리 용량 단위로 비교한다.

실제 byte 예산에는 인덱스·버퍼도 필요하다. 기대값은 RAM 수용의 확정 보장이 아니다.

한 작업 안에서 triangle을 찾는 방법

간선 (u, v) 의 양 끝점에 공통으로 연결된 정점을 찾는다

u 의 이웃

v 의 이웃

$\{v, a, b\}$

$\{n, b\}$

공통 이웃 b

발견한 삼각형: (u, v, b)

차수가 작은 정점을 먼저 두고, 동률이면 ID로 순서를 정한다.
그 순서에 맞는 이웃만 탐색한다. 원래 입력은 무방향 그래프다.

PTECD: 필요한 색의 이웃만 비교

색 순서가 0,1,2인 삼각형: 세 번째 정점은 색 2여야 한다

처리 방식	u에서 비교할 이웃	v에서 비교할 이웃
PTEBASE	{v, a, b}	{n, b}
PTECD	색 2의 이웃만: {b}	색 2의 이웃만: {b}

$u=0$, $v=1$, $a=1$, $n=0$, $b=2$ 는 각 정점의 색이다.

비교 대상은 줄어도 공통 이웃 b 와 삼각형 (u,v,b) 는 그대로다.

CD=color-direction. (u,v) 는 $E_star\{0,1\}$, 이웃은 $E_star\{0,2\} \cdot E_star\{1,2\}$ 에서 읽는다.

세 색의 순서 6가지를 각각 처리하므로 다른 방향의 삼각형도 빠뜨리지 않는다.

PTECD: Type-1 계산은 한 작업에만 배정

모든 정점의 색이 0인 삼각형을, 색 수 $p = 4$ 에서 생각해 보자

처리 방식	색 0 내부 삼각형을 계산할 작업	계산 / 최종 출력
PTEBASE	(0,1), (0,2), (0,3)에서 반복	계산 3회 / 출력 1회
PTECD	(0,1)에서만 계산	계산 1회 / 출력 1회

색별 담당 작업: 0은 (0,1), 1은 (1,2), 2는 (2,3), 3은 (0,3).
담당 작업에서만 $E_star[c,c]$ 로 색 c 내부 삼각형을 찾는다.

출력 중복만 막는 것에서, 같은 삼각형을 다시 찾는 계산까지 없애는 것이다.
다른 두 색 작업에서도 Type-2를 찾는 데 필요한 내부 간선은 계속 읽고 사용한다.

PTESC: 파일 4개는 유지하고, 1개만 교체

0→2 파일이 필요한 삼각형을 먼저 모두 찾고, 반대 방향을 처리한다

진행 순서	RAM에 두는 파일	동시 파일 수
1. 0→2를 쓰는 계산	공통 4개 + E_star{0,2}	4 + 1 = 5개
2. 끝난 파일 해제	E_star{0,2}를 RAM에서 내림 공통 4개는 계속 유지	4개
3. 2→0을 쓰는 계산	E_star{2,0}을 새로 읽음 공통 4개 + E_star{2,0}	4 + 1 = 5개

공통 4개: 색 0과 1 사이, 색 1과 2 사이의 방향별 파일 각 2개.

한 세 색 작업에서 총 6개를 한 번씩 읽지만, RAM에 동시에 두는 파일은 최대 5개다.

PTESC: 동시 적재 감소가 읽기 감소로 이어지는 이유

6개 파일은 모두 읽는다. 줄어드는 것은 색 수와 작업 간 반복 읽기다

1	동시 적재량	CD: 기대상 $6m/\rho^2$ SC: 기대상 $5m/\rho^2$
2	같은 RAM M	CD: $\rho = \text{ceil}(\text{sqrt}(6m/M))$ SC: $\rho = \text{ceil}(\text{sqrt}(5m/M))$
3	반복 읽기	$E_{\text{star}}[0,1]$ 은 두 색 작업 $(0,1)$ 과 세 색 작업 $(0,1,k)$ 에서 읽힌다: 총 $\rho - 1$ 개 작업

색 수가 실제로 줄면 읽는 작업 수도 줄어든다. 올림 때문에 같은 ρ 가 될 수도 있다.
기대 메모리 기준이며, 여러 작업 사이의 캐시 공유를 가정한 개선은 아니다.

TTP·CTTP와 PTE의 차이

비슷한 부분 문제를 풀지만, 간선을 모으는 방식이 다르다

비교 항목	TTP / CTTP	PTE
부분 문제 준비	각 작업에 필요한 간선을 map output으로 복제	색상쌍별 파일을 저장하고 필요한 작업이 직접 읽음
전체 shuffle	간선 수와 분할 수에 따라 함께 증가	입력 간선 수에 비례 $O(m)$
열거 단계	복제 간선을 shuffle한 뒤 reduce에서 처리	분할 파일을 읽은 뒤 map task 안에서 처리
남는 비용	shuffle의 임시 I/O·정렬 CTTP의 반복 입력 읽기	분할 파일의 반복 읽기 RAM 용량과 실제 열거 계산

$O(m)$ 은 shuffle 데이터량이다. 전체 실행시간이나 전체 통신량을 뜻하지 않는다.

논문의 실험 결과

2016년 실험: 40개 worker, 머신당 32 GB RAM

약 3.06조

ClueWeb12 전체 triangle 열거 성공

63억 정점, 720억 간선. 비교 대상들은 전체 그래프 처리에 실패.

최대 47배

PTESC의 실행시간 개선

SubDomain에서 CTTTP와 비교. 모든 데이터셋에 공통인 배수는 아님.

최대 245배

PTE의 shuffle 데이터량 감소

ClueWeb12 표본 약 360억 간선 조건에서 CTTTP와 비교.

CTTP는 200억 간선을 넘는 표본에서 5일 내 완료하지 못했다.

처리 실패의 원인은 방법별로 메모리 부족, ID 범위, 실행시간 한계 등으로 다르다.

정리와 적용 범위

PTE는 중간 데이터를 만드는 방식부터 바꾼다

PTEBASE	간선을 미리 분할해 대량의 shuffle을 줄임
---------	----------------------------

PTECD	필요한 이웃만 비교하고 중복 계산을 제거
-------	------------------------

PTESC	계산·loading 순서를 조정해 network read를 줄임
-------	-------------------------------------

적용 범위: 논문이 다루는 정적·무방향·단순 그래프의 정확한 열거.

남는 제약: 작업별 RAM, 분산 저장공간, 통신, 그리고 발견할 삼각형 수.

부록 A. 의사코드로 보는 전체 흐름

설명용 요약: 정확한 branch와 순서는 원문 Algorithms 1-4 참조

준비: 정점에 차수·ID 순서를 부여하고, 각 정점의 색을 정한다.

분할: 각 ordered edge (u, v) 를 key $(\text{color}(u), \text{color}(v))$ 로 묶어 저장한다.

열거: 두 색 또는 세 색 조합의 작업을 병렬로 실행한다.

각 작업은 필요한 방향별 간선 파일을 RAM으로 읽는다.

가능한 색상·정점 순서의 경우들을 빠짐없이 처리한다.

pivot (u, v) 마다 허용된 u 이웃과 v 이웃의 교집합을 구한다.

공통 이웃 w 를 찾으면 $\text{enum}(u, v, w)$ 를 호출한다.

Type 1은 지정된 부분 문제에서만 열거한다.

PTESC: 세 색 작업의 내부 호출 순서를 조정해 최대 5개 묶음을 유지한다.

색상쌍의 순서를 min/max로 바꾸지 않는다.

방향별 $E_star[i, j]$ 와 $E_star[j, i]$ 를 구분해야 color-direction을 활용할 수 있다.

부록 B. 비용과 메모리 보장의 범위

측정 대상	GP / TTP / CTP	PTE
Shuffle 데이터량	$O(m^{3/2} / \sqrt{M})$	$O(m)$
PTE의 network read	이 표에서는 비교 생략	$O(m^{3/2} / \sqrt{M})$
PTE의 total work	이 표에서는 비교 생략	$O(m^{3/2})$

m 은 간선 수. M 은 한 작업이 사용할 수 있는 메모리의 이론적 규모다.
확률적 색상 배정은 기대 크기를 조절한다. 모든 부분 문제의 크기가 항상
같거나, RAM 초과 가능성이 사라진다는 보장과는 구분해야 한다.

같은 색 라벨의 정점 수나 간선 수는 실행마다 달라질 수 있다.
고차수 정점에 연결된 간선들은 동일한 정점 색을 공유한다.

부록 C. 기대 크기가 작아도 실제로 넘칠 수 있다

중심 c 에 네 앞이 연결된 별 모양 그래프: 간선 4개, triangle 0개

색 배정 예시	읽는 간선 묶음	실제 간선 수
$c=0, a=b=1$	E01: $(c,a), (c,b)$	2개
$d=e=2$	E02: $(c,d), (c,e)$ E12: 없음	2개 + 0개

$M=3$ 이면 $\rho = 3$. 세 색 작업의 기대 크기 $= 8/3 \leq 3$.

하지만 위 색 배정에서는 $2+2+0 = 4 > 3$ 이다.

기대값만으로 매 실행의 메모리 수용을 증명할 수 없다는 수학적 예시다.

실제 PTE 실행 실패를 측정한 결과가 아니며, triangle 수와 입력 적재량도 다르다.